CÓDIGOCLARO · RUTA JAVA

Tema 14 · Genéricos y lambdas

Guía de estudio · DAM/DAW

Tipos parametrizados, Comparator, streams y expresiones lambda.

1. Explicación

Los genéricos expresan el tipo de elementos y detectan incompatibilidades en compilación. Una lambda implementa una interfaz funcional con un método abstracto. Stream permite filtrar, transformar y consumir datos; una operación terminal consume el stream, que no se puede reutilizar.

2. Ejemplo

Código · java

```
import java.util.Comparator;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<String> palabras = List.of("sol", "cielo", "mar");
        List<String> largas = palabras.stream()
            .filter(p -> p.length() >= 4)
            .sorted(Comparator.comparingInt(String::length).thenComparing(String::compareTo))
            .toList();
        System.out.println(largas);
    }
}
```

Resultado esperado

```
[cielo]
```

3. Errores frecuentes

- Usar tipos raw y perder comprobación estática.
- Escribir lambda incompatible con la interfaz funcional esperada.
- Reutilizar el stream después de una operación terminal.

4. Ejercicio

Filtra palabras de cuatro o más letras y ordénalas.

Casos que debe superar

Entrada	Resultado esperado
["sol","cielo","mar"]	[cielo]
["lago","nube"]	[lago, nube]
[]	[]

5. Solución explicada

Código · java

```
import java.util.*;
public class Main {
    static List<String> filtrar(List<String> palabras) {
        return palabras.stream().filter(p -> p.length() >= 4).sorted().toList();
    }
    public static void main(String[] args) { System.out.println(filtrar(List.of("sol","cielo",
"mar"))); }
}
```

Otras soluciones válidas

- Usar un bucle si un flujo declarativo oculta pasos relevantes.
- Usar Comparator.comparing para ordenar según una propiedad.

6. Fuentes oficiales

Dev.java — Writing Your First Lambda Expression · Dev.java — The Stream API